AI Engineering and Claude Architect · Part III, Agents

Agents and MCP, Lesson 8

Hooks, Enforcement, Escalation and Humans in the Loop

The system prompt said, in capitals, "VERIFY IDENTITY BEFORE ANY REFUND." It worked in every test. Then, across a month of production traffic, a handful of refunds went out with no verification at all. The prompt had lowered the risk. It had never removed it.

Module	AI06 of 13
Lesson	8 of 10
Prerequisites	AI06 L2–L3, L5
Exam relevance	**Direct.** All of TS 1.4, TS 1.5 and TS 5.2, and much of Scenario 1
Reading time	~45 minutes

Contents

1. Hooks, Enforcement, Escalation and Humans in the Loop

Lesson 8 of 10 · Agents and MCP

Hooks, Enforcement, Escalation and Humans in the Loop

Key idea

A prompt guides, and code guarantees. If a rule must hold every time, such as identity before a refund, or a refund limit, put it in a hook or a gate that runs whatever the model decides.

Escalate on clear triggers, not on sentiment or the model's own confidence, and hand over a structured summary, since the human can't see the transcript.

> *Before: the loop, tool design, tool errors. Teaches: PreToolUse and PostToolUse hooks, prerequisite gates, data normalization, escalation triggers, handoff. Used by: AI06 L9, AI09, AI12 L3, AI13.*

The hook: what a prompt can't promise

Every model call is probabilistic (AI02 L5). A rule stated in a prompt has a failure rate. It might be tiny, but it isn't zero, as the guide says.

Illustrative figures	Per month
Refund conversations	30,000
If the model skips verification 1% of the time	**300 unverified refunds**
At 0.1%	**30**
With a code gate on that rule	**0**

The rates are invented, to show scale. The zero isn't: code runs every time, provided the gate itself is correct and all refunds pass through it.

A gate in action. Real output from `code/ai06-agents/.../PolicyHooks.java`:

```
Refund before verification: Decision[action=DENY, reason=Call get_customer and verify c-19 first, redirect=]
Refund of $120 after:       Decision[action=ALLOW, reason=, redirect=]
Refund of $900 after:       Decision[action=REDIRECT, reason=Refunds over $500 need a human,
redirect=escalate_to_human]
```

Intuition: the speed limiter and the sign

A road sign says "Maximum 60." Most drivers obey. A speed limiter on a truck's engine means that it physically can't go faster.

A prompt is the sign, and a hook is the limiter. Signs are fine for "drive considerately." You want a limiter on the thing that can't ever happen. The sign still has a use: it tells the driver why, so they plan around it. So keep the rule in the prompt too, which reduces how often the limiter has to intervene.

Mechanism: where hooks sit

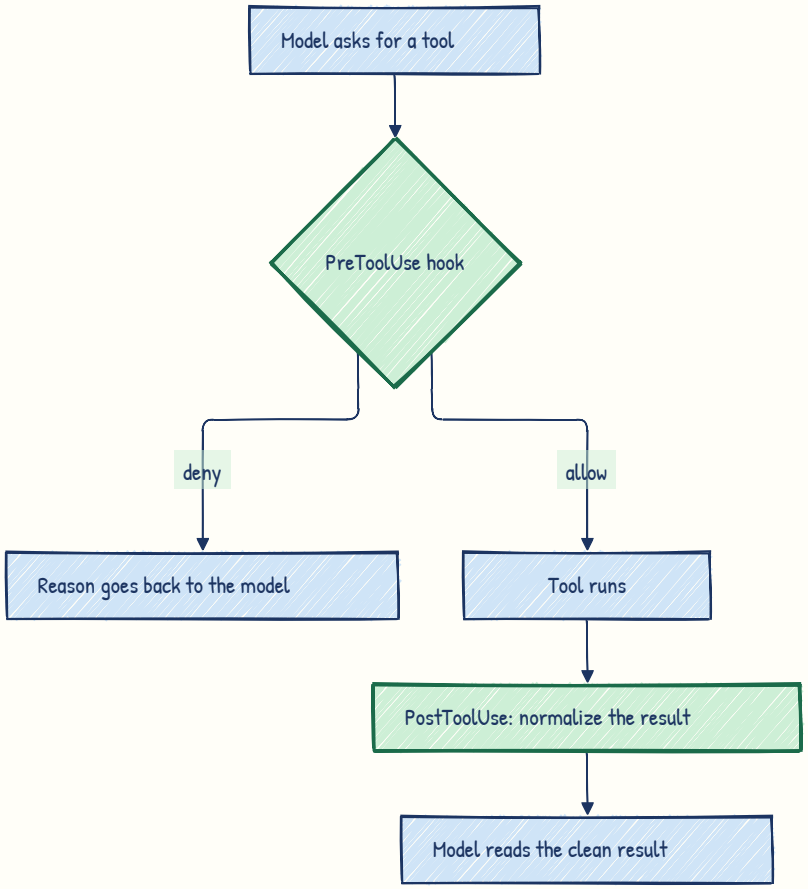


Figure 1. Hooks around a tool call. PreToolUse can block or change a call before it runs. PostToolUse can change the result before the model reads it.

Hooks run in your application, outside the model. The model can't talk its way past them, and retrieved text can't inject its way past them.

What each hook does

Hook	Runs	Use for	Exam example
PreToolUse	Before the tool	Block, redirect or rewrite calls	Block refunds over $500, redirect to escalation
PostToolUse	After the tool	Normalize, redact, add context	Unix timestamps and status codes to one format

Normalizing data with PostToolUse

Different MCP tools return different formats. The model might cope, or might misread them, and every extra format is another chance to go wrong. Real output:

```
Tool A raw:   {updated_at=1762300800, status=2}
Model reads:  {status=SHIPPED, updated_at=2025-11-05T00:00:00Z}
Tool B raw:   {updated_at=2025-11-05T00:00:00Z, status=SHIPPED}
Model reads:  {status=SHIPPED, updated_at=2025-11-05T00:00:00Z}
```

Prerequisite gates

A gate blocks a downstream tool until an earlier step has succeeded. It's a small state machine.

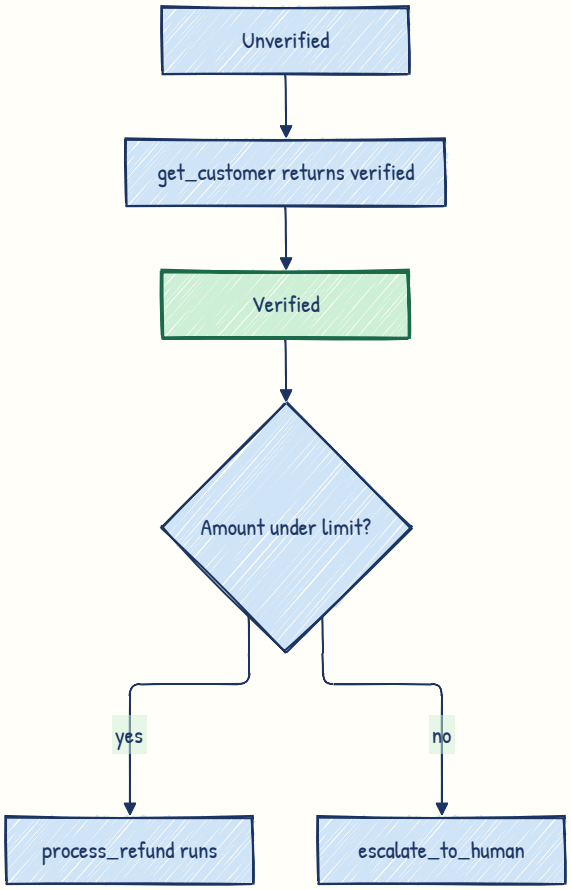


Figure 2. A prerequisite gate. process_refund stays blocked until get_customer has returned a verified customer, whatever order the model tries.

Note what's checked: **verified state recorded by code from the tool's result**, not the model's claim that it verified.

Hooks in Claude Code and the Agent SDK

From the hooks reference. Three levels of nesting: event, matcher group, handler.

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "mcp__billing__process_refund",
        "hooks": [
          {
            "type": "command",
            "command": "node",
            "args": ["${CLAUDE_PROJECT_DIR}/.claude/hooks/check-refund.js"],
            "onFailure": "block"
          }
        ]
      }
    ]
  }
}
```

A command hook gets JSON on stdin, including `tool_name` and `tool_input`. To deny:

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "Refunds over $500 need a human. Use escalate_to_human."
  }
}
```

Detail	What to know
`permissionDecision`	`allow`, `deny`, `ask` or `defer`
`updatedInput`	Replaces the tool's arguments before it runs
Silence	Exit 0 with no output means no decision, so **a hook can deny, but silence doesn't approve**
Exit code 2	Blocks, on events that can block. stderr becomes the reason
Exit code 1	**Doesn't block!** It's a non-blocking error, and the action proceeds
`onFailure: "block"`	A crash, timeout or missing script blocks, rather than letting it through
PostToolUse	`hookSpecificOutput.updatedToolOutput` replaces a result, and `additionalContext` adds a note
Handler types	`command`, `http`, `mcp_tool`, `prompt`, `agent`

A security hook that fails open

A policy hook that crashes with exit 1 lets the action through. For a rule that must hold, set `onFailure: "block"`, and test the failure path. Also remember that a `prompt` handler uses a model, so it's not deterministic: use `command` or `http` for guarantees.

Escalation: when to hand over

TS 5.2 gives clear triggers, and names the wrong ones.

Trigger	Example
The customer asks for a human	"I want to talk to a person"
A policy gap	The question isn't covered by policy
No progress	Repeated attempts haven't moved it forward

Not a good trigger	Why
Sentiment	An angry customer may have an easy problem, and a calm one may have a hard one. It measures the wrong thing
Self-reported confidence	Uncalibrated
Heuristic matching across customers	With several matches, **ask for another identifier**

Real output:

```
I want to speak to a human         -> customer asked for a human
This is ABSOLUTELY awful!          -> none
Do you price-match competitors?    -> policy gap
```

The second line is deliberate. Anger alone doesn't escalate. The agent should acknowledge it and try to solve the problem, and if it asks for a human, or no progress is being made, the other triggers apply. This keyword check is only an illustration: in a real system the model usually decides, using criteria in the prompt, with hard rules in code.

Handoff: the human sees nothing else

The guide says human agents lack access to the conversation transcript, so the summary has to stand alone.

Field	Example
Customer ID	`c-19` (verified)
Root cause	Order 8120 charged twice
Amount	$900.00
Recommended action	Approve, since both charges are confirmed
Already tried	Verified customer, refunded $120 on order 7781, $900 blocked by limit

A bad handoff says "customer is upset about a refund." The human then starts again from the beginning, and the customer repeats everything.

Multi-concern requests

"My parcel's late, and I was charged twice." **TS 1.4** says to decompose it into items, investigate each using shared context, and combine them into one response. Don't answer only the first thing mentioned.

Humans in the loop

Pattern	When	Example
Approve before action	Hard to undo	Large refund, sending external email
Review after action	Easy to undo, high volume	Sampled refunds
Escalate on trigger	Beyond agent's remit	Policy gap
Elicitation in MCP	A server needs input	Confirm a destructive action

Too many prompts cause fatigue, and people start approving without reading. AI09 L3 covers that.

Why the obvious fixes fall short

Tempting move	Why it falls short	Better
Stricter, louder prompt	Lowers the rate, and never reaches zero	Hook or gate
More few-shot examples of the rule	Same	Hook or gate
Model checks its own compliance	Same weakness twice	Code
Sentiment analysis for escalation	Wrong signal	The three triggers
"Escalate if confidence below 0.7"	Uncalibrated	The three triggers
Pass the raw transcript to a human	Long, and the guide says they lack access	Structured summary
Pick the likeliest customer among matches	Could expose someone's account	Ask for another identifier

Prompts still matter. Use them for judgment: tone, how to explain, what's worth asking. Use code for guarantees.

How Anthropic approaches it

Layer	Mechanism	Source
Agent SDK and Claude Code	Hooks	Hooks reference
Tool Runner	Gate inside the tool's function, or inspect each yielded message	Claude API docs
Managed Agents	Permission policies: `always_allow`, `always_ask`, `auto`	Claude API docs
Protocol	"SHOULD always be a human in the loop with the ability to deny"	MCP tools spec
Dedicated tools	A `send_email` tool is easy to gate, and `bash -c "curl ..."` isn't	Claude API agent design guide

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: rules in the prompt. It's easy, and it's probabilistic.

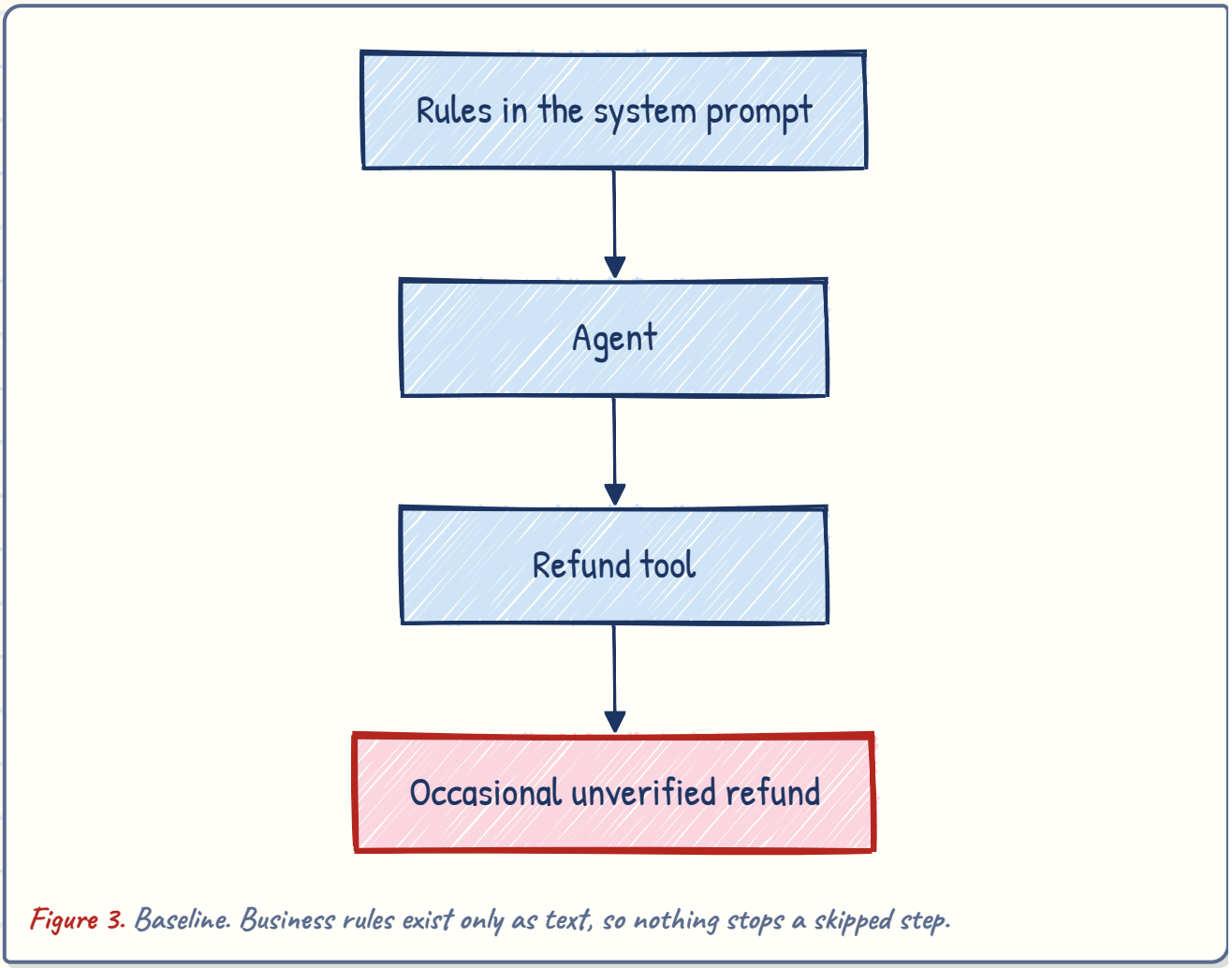


Figure 3. Baseline. Business rules exist only as text, so nothing stops a skipped step.

Improved: hooks in the agent process. Rules hold. New problem: rules are duplicated across agents, and anything calling the backend directly bypasses them.

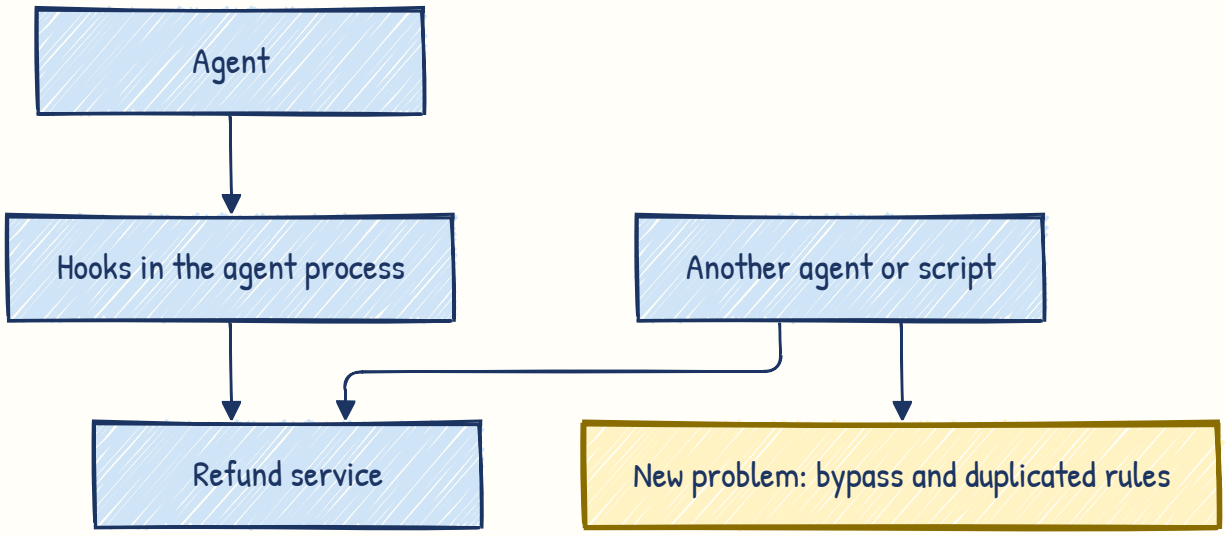


Figure 4. Improved. Hooks enforce rules for this agent, while other agents and callers can still reach the backend without them.

Production: defense in depth.

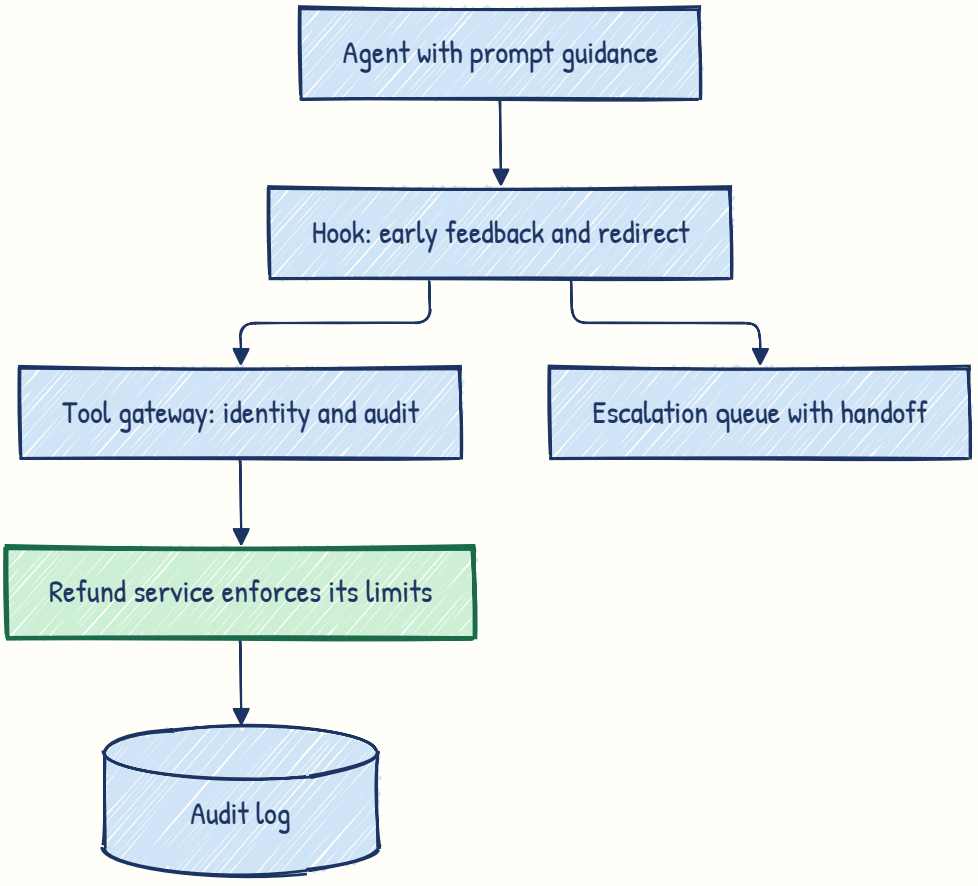


Figure 5. Production. The hook gives fast feedback to the model, but the refund service itself enforces limits, so no caller can bypass them. Every decision is audited.

Component	Why it exists	How it communicates
Prompt	Good behavior most of the time, and explains the rules	Text
Hook	Instant, explainable feedback to the model	In-process, or local command
Service limits	**The real boundary**	The backend rejects, whatever the caller
Audit log	Proof, and evals	Async
Escalation queue	A home for handoffs	Message queue, with structured payload

Failures

Failure	Containment
Hook crashes	`onFailure: "block"`, and an alert
Hook bug blocks everything	Tests in CI, and a rollback path
Normalizer mangles a format	Keep the raw value in logs
Queue backs up	Tell the customer the wait, and monitor
Agent loops after a denial	The denial says what to do instead, with a repeat detector

Java

From `PolicyHooks.java`. The gate reads state set by code, and returns a reason that the model can act on.

```
if (!verifiedCustomers.contains(customer)) {
    return Decision.deny("Call get_customer and verify " + customer + " first");
}
double amount = Double.parseDouble(call.input().get("amount"));
if (amount > REFUND_LIMIT) {
    return Decision.redirect("Refunds over $500 need a human", "escalate_to_human");
}
```

State is set in the post hook, only from a real tool result:

```
if (call.tool().equals("get_customer") && "true".equals(result.get("verified"))) {
    verifiedCustomers.add(call.input().get("customer_id"));
}
```

Real code needs per-conversation state, and money as `BigDecimal`. The Agent SDK's hook API is for Python and TypeScript, so a Java service puts this in its own tool dispatcher (AI08 L3).

Trade-offs

Decision	Gain	Cost
Hook over prompt	Guarantee	Code to maintain, less flexibility
Service-side limits	Nobody can bypass them	Slower to change
Approve before action	Safety	Latency and fatigue
Normalizing	Consistency	Must keep up with new formats
Escalating more	Fewer agent errors	Higher human cost

When a prompt is enough: tone, formatting, preferences, anything where the occasional miss is harmless.

Common mistakes

1. **Using a prompt for a guaranteed rule.** It has a non-zero failure rate.
2. **Trusting the model's claim that it verified.** Record it from the tool.
3. **Letting a policy hook fail open.** Exit 1 doesn't block.
4. **Relying only on the agent hook.** Enforce in the service too.
5. **Escalating on sentiment.** It measures the wrong thing.
6. **Escalating on self-reported confidence.** It's uncalibrated.
7. **Guessing between matching customers.** Ask for another identifier.
8. **Vague handoffs.** The human can't see the transcript.

What to say in an interview

"Guaranteed rules go in code, since prompts have a non-zero failure rate. I'd use a PreToolUse hook to block refunds until identity is verified, and to redirect anything over the limit to a human, with the backend enforcing limits as well. A policy hook must fail closed. I'd escalate when a customer asks, on a policy gap, or on no progress, and never on sentiment, and I'd hand over customer ID, root cause, amount and a recommendation."

Certification lens

How this lesson maps to CCAR-F

TS 1.4, **TS 1.5** and **TS 5.2.** Guaranteed means code. Judgment means prompt. This is among the guide's most repeated habits. Hook field details are unlikely to be tested, but `PostToolUse` for normalizing and call interception for blocking are named.

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Customer Support Resolution Agent · Medium. Refunds must never be processed before identity verification. The system prompt says so, but a few slip through. What should you do?

A. Repeat the instruction at the end of the system prompt.

B. Add few-shot examples showing verification first.

C. Ask the model to confirm that it verified before refunding.

D. Add a prerequisite gate that blocks `process_refund` until `get_customer` has returned a verified customer.

A D (TS 1.4). A, B and C all stay probabilistic.

Q Q2 · Exam style, scenario: Customer Support Resolution Agent · Medium. Refunds over $500 must go to a human. Which is the best implementation?

A. A hook that intercepts `process_refund`, blocks amounts over $500, and redirects to the escalation workflow.

B. A prompt rule with the $500 limit in bold.

C. A sentiment check that escalates upset customers.

D. A post-processing report of large refunds, reviewed weekly.

A **A** (TS 1.5). B is probabilistic. C measures something else. D finds the problem after the money has gone.

Q Q3 · Exam style, scenario: Multi-Agent Research System · Medium. MCP tools return dates as Unix timestamps, ISO 8601 and locale strings, and the agent sometimes misorders events. What's the best fix?

A. Tell the agent in the prompt how to convert all the formats.

B. Add a `PostToolUse` hook to normalize dates before the model sees them.

C. Replace all the MCP servers.

D. Add a tool for converting dates.

A **B** (TS 1.5). A is probabilistic. C is out of proportion. D adds calls, and depends on the model using it.

Q Q4 · Exam style, scenario: Customer Support Resolution Agent · Medium. A customer is using angry language, but the issue is a simple address change that the agent can do. They haven't asked for a human. What should the agent do?

A. Escalate at once, because of the sentiment.

B. Ask the customer to calm down.

C. Acknowledge the frustration, and resolve the address change.

D. Lower its confidence score, and escalate if it falls below 0.7.

A **C** (TS 5.2). A and D use wrong triggers. B makes things worse.

Q Q5 · Exam style, scenario: Customer Support Resolution Agent · Hard. `get_customer` returns two customers with the same name. Choose the **two** correct actions, plus the correct way to escalate if that's needed later.

A. Pick the most recently active one.

B. Ask for another identifier, such as an email or order number.

C. Don't take account actions until the customer is identified.

D. Refund both, to be safe.

E. Pick by matching the sentiment of past tickets.

A **B and C** (TS 5.2). A and E are heuristic guesses that can expose another person's account. D is plainly wrong. If it's escalated later, hand over a structured summary with customer details, root cause, amount and recommended action.

Revision - Lesson 8

→ Prompts guide, and code guarantees. Prompts have a non-zero failure rate.

→ PreToolUse blocks, redirects or rewrites. PostToolUse normalizes results.

→ Gates block a tool until a prerequisite really succeeded.

→ Record verified state from the tool's result, not the model's claim.

→ Exit 2 blocks. Exit 1 doesn't. Set onFailure to block for policy hooks.

→ Silence isn't approval, and a prompt handler isn't deterministic.

→ Enforce limits in the backend service too.

→ Escalate when asked, on a policy gap, or on no progress.

→ Not on sentiment, not on self-reported confidence, and not by guessing among matches.

→ Multiple matches: ask for another identifier.

→ Handoff: customer ID, root cause, amount, recommended action.

→ Decompose multi-concern requests, and give one unified answer.

Your notes